

Claude Code Cheatsheet

Daily essentials for maximum productivity

Florian BRUNIAUX

July 2026

v1.0.0

Guide v3.41.1



Table of contents

1	How to Install	2
2	Essential Commands	4
3	Keyboard Shortcuts	6
4	File References	7
5	Lesser-Known Features (But Official!)	8
6	Permission Modes	9
7	Memory & Settings (2 levels)	10
7.1	.claude/ Folder Structure	10
8	Context Management (CRITICAL)	11
8.1	Context Recovery Commands	13
9	Plan Mode & Thinking	14
10	Typical Workflow	15
11	Quick Prompting Formula	16
12	MCP Servers	17
13	Search Tools Quick Reference	18
14	CLI Flags Quick Reference	19
15	Anti-patterns	20
16	Cost Optimization	21
17	Quick Decision Tree	22
18	Common Issues Quick Fix	23

1 How to Install

```
npm install -g @anthropic-ai/claude-code # Verify installation which claude && claude --version
```

Health Check: `claude doctor && claude mcp list`

2 Essential Commands

Command	Action
<code>/help</code>	Contextual help
<code>/powerup</code>	Interactive animated lessons teaching Claude Code features
<code>/clear</code>	Reset conversation
<code>/compact</code>	Free up context
<code>/status</code>	Session state + context usage
<code>/context</code>	Detailed token breakdown
<code>/plan</code>	Enter Plan Mode (no changes)
<code>/ultraplan</code>	Cloud Plan Mode: draft in cloud, review in browser
Approve the plan	Exit Plan Mode (<code>Shift+Tab</code> to leave without approving)
<code>/model</code>	Switch model (sonnet/opus/opusplan)
<code>/insights</code>	Usage analytics + optimization report
<code>/simplify</code>	Detect over-engineering in changed code + auto-fix
<code>/batch</code>	Large-scale refactors via 5-30 parallel worktree agents
<code>/teleport</code>	Teleport session from web
<code>/tasks</code>	Monitor background tasks
<code>/remote-env</code>	Configure cloud environment
<code>/rc</code>	Start remote control session (Research Preview, Pro/Max)
<code>/mobile</code>	Get Claude mobile app download links
<code>/fast</code>	Toggle fast mode (2.5x speed, 6x cost)
<code>/voice</code>	Toggle voice input (hold Space to speak)

3 Keyboard Shortcuts

Shortcut	Action
<code>Shift+Tab</code>	Cycle permission modes
<code>Esc × 2</code>	Rewind (undo)
<code>Ctrl+C</code>	Interrupt
<code>Ctrl+R</code>	Search command history
<code>Ctrl+L</code>	Clear screen (keeps context)
<code>Ctrl+B</code>	Background tasks
<code>Ctrl+F</code>	Kill all background agents (double press)
<code>Alt+T</code>	Toggle thinking on/off
<code>Space</code> (hold)	Voice input (requires <code>/voice</code>)
<code>Tab</code>	Autocomplete
<code>Shift+Enter</code>	New line
<code>Ctrl+D</code>	Exit

IDE Shortcuts: VS Code `Alt+K` | JetBrains `Cmd+Option+K`

4 File References

@path/to/file.ts → Reference a file @agent-name → Call an agent !shell-command → Run shell command

5 Lesser-Known Features (But Official!)

Feature		What It Does
Tasks API		Persistent task lists with dependencies
Background Agents		Sub-agents work while you code
Agent Teams		Multi-agent coordination (TeamCreate/SendMessage)
Auto-Memories		Automatic cross-session context capture
Session Forking		Rewind + create parallel timeline
LSP Tool		Code intelligence (go-to-def, refs), ~50ms vs 45s with grep
Voice Mode		Native voice input, free transcription
Remote Control		Control local session from phone/browser (Research Preview, Pro/Max)
Cloud/Desktop Tasks	Scheduled	Machine-off scheduling via <code>/schedule</code> or Desktop app
Skill Evals		A/B testing and trigger tuning for custom skills
Output Styles		<code>/config</code> → Default, Explanatory, Learning modes
Rules Templates		Ready-to-use <code>.claude/rules/</code> templates (arch/quality/test/perf)

Pro tip: These aren't "secrets", they're in the [CHANGELOG](#). Read it!

6 Permission Modes

Mode	Editing	Execution
Default	Asks	Asks
acceptEdits	Auto	Asks
Plan Mode	None	None
auto	Classifier decides	Classifier decides
dontAsk	Only if in allow rules	Only if in allow rules
bypassPermissions	Auto	Auto (CI/CD only)

Shift+Tab to switch modes

7 Memory & Settings (2 levels)

Level	Path	Scope	Git
Project	<code>.claude/</code>	Team	Yes
Personal	<code>~/.claude/</code>	You (all projects)	No

Priority: Project overrides Personal

7.1 .claude/ Folder Structure

```
.claude/ |— CLAUDE.md      # Local memory (gitignored) |— settings.json  # Hooks (committed)
|— settings.local.json # Permissions (not committed)
|— agents/         # Custom agents |— hooks/         # Event scripts |— rules/         # Auto-loaded rules
| |— architecture-review.md | |— code-quality-review.md | |— test-review.md
| |— performance-review.md | |— skills/         # Slash commands + knowledge modules (unified)
```

8 Context Management (CRITICAL)

Statusline: Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%

✔ 0-50%	⚠ 50-70%	💎 70-90%	🏆 90%+

Watch `Ctx(u):` → >70% = `/compact` , >85% = `/clear`

Sign	Action
Short responses	<code>/compact</code>
Frequent forgetting	<code>/clear</code>
>70% context	<code>/compact</code>
Task complete	<code>/clear</code>

8.1 Context Recovery Commands

Command	Usage
<code>/compact</code>	Summarize and free context
<code>/clear</code>	Fresh start
<code>/rewind</code>	Undo recent changes
<code>claude -c</code>	Resume last session
<code>claude -r <id></code>	Resume specific session

9 Plan Mode & Thinking

Feature	Activation	Usage
Plan Mode	<code>Shift+Tab × 2</code> or <code>/plan</code>	Explore without modifying
OpusPlan	<code>/model opusplan</code>	Opus for planning, Sonnet for execution
Ultraplan	<code>/ultraplan <prompt></code>	Cloud planning, browser review, terminal stays free

Opus 4.8: Default effort is **xhigh**. Use `ultrathink` to force max effort for the next turn.

Control	Action	Persistence
Alt+T	Toggle thinking on/off	Session
/config	Enable/disable globally	Permanent
<code>/effort</code> slider	low/medium/high/xhigh/max	Session
<code>effort</code> param	API only, per-request	Per-request

Cost tip: For simple tasks, Alt+T to disable thinking → faster & cheaper.

10 Typical Workflow

- | | | | | | |
|------------------|--------------------------|-------------------|-------------------------|--------------------------|--|
| 1. Start session | → <code>claude</code> | 2. Check context | → <code>/status</code> | 3. Plan Mode | → <code>Shift+Tab</code> × 2 (for complex tasks) |
| 4. Describe task | → Clear, specific prompt | 5. Review changes | → Always read the diff! | 6. Accept/Reject | → <code>y/n</code> |
| 7. Verify | → Run tests | 8. Commit | → When task complete | 9. <code>/compact</code> | → When context >70% |

11 Quick Prompting Formula

WHAT: [Concrete deliverable] WHERE: [File paths] HOW: [Constraints, approach] VERIFY: [Success criteria]

Example:

Add input validation to the login form. WHERE: src/components/LoginForm.tsx
HOW: Use Zod schema, show inline errors VERIFY: Empty email shows error, invalid format shows error

12 MCP Servers

Server	Purpose
Serena	Indexing + session memory + symbol search
grepai	Semantic search + call graph analysis
Context7	Library documentation
Sequential	Structured reasoning
Playwright	Browser automation
Postgres	Database queries
doobidoo	Semantic memory + Knowledge Graph

Serena memory: `write_memory()` / `read_memory()` / `list_memories()`

Check status: `/mcp`

13 Search Tools Quick Reference

Quick decision: exact text → `rg` | exact name → `rg` /Serena | concept → `grepai` | structure → `ast-grep`

Task	Tool
Find exact text	<code>rg "TODO"</code>
Find auth-related code	<code>grepai search "authentication"</code>
Who calls a function	<code>grepai trace callers "login"</code>
Get file structure	<code>serena get_symbols_overview</code>

14 CLI Flags Quick Reference

Flag	Usage
<code>-p "query"</code>	Non-interactive mode (CI/CD)
<code>-c</code> / <code>--continue</code>	Continue last session
<code>-r</code> / <code>--resume <id></code>	Resume specific session
<code>--teleport</code>	Teleport session from web
<code>--model sonnet</code>	Change model
<code>--add-dir ../lib</code>	Allow access outside CWD
<code>--permission-mode plan</code>	Plan mode
<code>--worktree</code> / <code>-w</code>	Run in isolated git worktree
<code>--max-budget-usd 5.00</code>	Max API spend limit (print mode)
<code>--dangerously-skip-permissions</code>	Auto-accept (use carefully)
<code>--mcp-debug</code>	Debug MCP servers
<code>--allowedTools "Edit,Read"</code>	Whitelist tools
<code>--debug</code>	Debug output

15 Anti-patterns

❌ Don't

- Vague prompts
- Accept without reading
- Ignore warnings
- Skip permissions
- Negative constraints only

✅ Do

- Specify file + line with @references
- Read every diff
- Use /compact at 70%
- Never in production
- Provide alternatives

16 Cost Optimization

Model	Use For	Cost
Haiku	Simple fixes, reviews	\$
Sonnet	Most development	\$\$
Opus	Architecture, complex bugs	\$\$\$
OpusPlan	Plan (Opus) + Execute (Sonnet)	\$\$

17 Quick Decision Tree

Simple task → Just ask Claude Complex task → Tasks API to plan first
Risky change → Plan Mode first Repeating task → Create agent or command Context full → /compact or /clear
Need docs → Use Context7 MCP Deep analysis → Use Opus (thinking on by default)

The Golden Rules

- 1 Always review diffs before accepting
- 2 Use /compact before context gets critical (>70%)
- 3 Be specific in requests (WHAT, WHERE, HOW, VERIFY)
- 4 Plan Mode first for complex/risky tasks
- 5 Create CLAUDE.md for every project
- 6 Commit frequently after each completed task
- 7 Know what's sent: prompts, files, MCP results → Anthropic

18 Common Issues Quick Fix

Problem	Solution
“Command not found”	Check PATH, reinstall npm global
Context too high (>70%)	<code>/compact</code> immediately
Slow responses	<code>/compact</code> or <code>/clear</code>
MCP not working	<code>claude mcp list</code> , check config
Permission denied	Check <code>settings.local.json</code>
Hook blocking	Check hook exit code, review logic

Health Check: `which claude && claude doctor && claude mcp list`

Author: Florian BRUNIAUX | [Methode Aristote](#) | Written with Claude
Version 3.41.1 | July 2026